Script 04

- Code organization and the scene graph.
- Perspective Camera vs Orthographic Camera.
- Animation: querying the scene graph and applying local transformations.
- Interaction: responding to keyboard and mouse events.
- Interaction: using interaction widgets.

1.1 Code organization and the scene graph

Open the folder **01_Ex_Code_Organization**.

Notice the following:

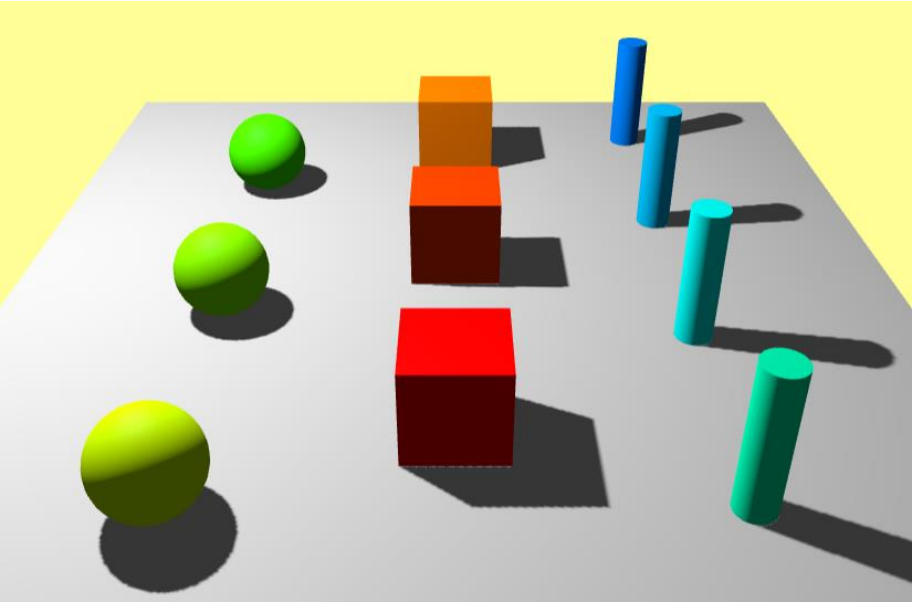
- The Javascript functions have been divided into **helper functions** – which stay mostly the same for different examples – and **scene modeling functions**.

- The **scene graph** stores the various scene models and establishes hierarchical relationships between them.

Tasks:

- Add **additional models** to the scene – create simple **"factory functions"** to maintain modular organization and facilitate development.

- Add **a second spotlight** to the scene; observe the illumination and shadow effects.



1.2 Animation – Local transformations

Open the folder **02_Ex_Animation**.

Analyze the changes made in the Javascript code that enable animating the movement of the spotlight.

Notice the following:

- The **computeFrame()** function computes and updates **transformation parameters** for the scene elements being animated.
- The **scene graph can be queried** to access scene elements.

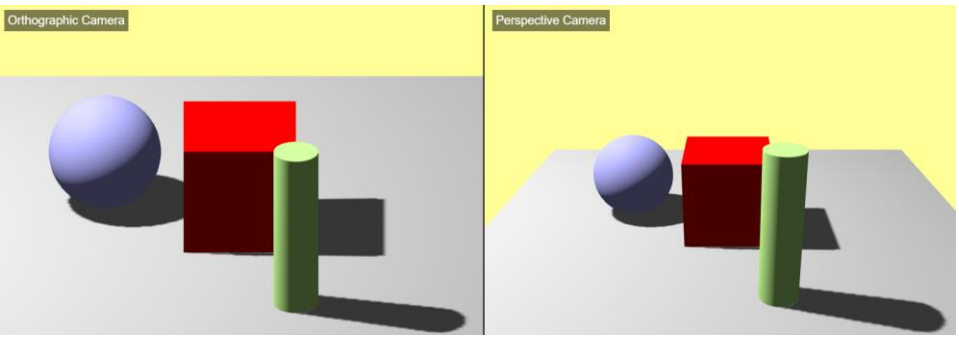
Tasks:

Add the following animation behavior:

- The **cylinder** rotates around its XX axis.
- The **cube** rotates around its YY axis.
- The **sphere** slides on the plane, along the ZZ direction.

1.3 Cameras

Open the folder **03_Cameras**.



The same scene is presented twice, side by side, using two different projections/cameras: on the left, **orthographic projection** and, on the right, **perspective projection**. Notice the differences. Which one looks more realistic?

Identify the **changes** made in the Javascript **code**: two cameras and two renderers, as well as updating the features of both cameras whenever an event occurs. The code defining the models and the scene is unchanged.

Tasks:

- Add other models to the scene, as done in the first example.
- Add more light sources to the scene.

1.4 Interaction – Keyboard

Open the folder **04_Ex_Interaction**.

Events are now being processed:

- See what happens when the **browser window is resized** – compare with the previous examples.
- Analyze how the **AWSD keys** are used to control the position of the cube.

Tasks:

- Use the **+ and – keys** to control the size of the sphere – suggestion: it should always touch the ground plane.
- Use the **cursor keys** to control the position of the cylinder.

Extra Task:

- In a similar way, use the **mouse buttons** to control model and scene features.

1.5 Interaction – GUI and widgets

Open the folder **05_Ex_GUI**.

It is possible to add **simple interaction widgets** and control model or scene features.

The widgets allow the user to input numerical values, choose a color, select/unselect a feature, etc.

Tasks:

- Identify the **code snippets** that **create the widgets** and allow the user to **input data** or **make a choice**.
-
- Add widgets to the previous example, to control model movements, etc.

1.6 Interaction – Camera control using the mouse

Open the folder **06_Ex_Interaction**.

Three.js offers different extensions to control the camera (rotation, pan, zoom, etc.) using the mouse.

We have already used **OrbitControls** in some examples from previous weeks. The user can easily control the camera and inspect a model or scene by orbiting around it:

- Left mouse button: scene rotation
- Right mouse button: scene pan
- Mouse scroll wheel: scene zoom

OrbitControls is **not directly accessible** from the three.js library and is loaded separately.

Task:

- Identify the **simple code additions** that were made to the initial example and now allow controlling the camera with the mouse.
- Check the documentation and see how you can **restrict camera movements**; for instance, to avoid looking below the ground plane.

Extra Task:

- In a similar way, try using some of the **other camera controls** available.